



CENTRO UNIVERSITÁRIO JORGE AMADO

**Curso Superior de Tecnologia em Análise e
Desenvolvimento de Sistemas**

Aluna: Victoria de Noronha dos Santos

Matrícula: 2240109472

Disciplina: Programação Orientada à Objetos - IL10311

**TEMA: DESENVOLVIMENTO DE PROJETO DE APLICAÇÃO
COM USO DE VETORES DE OBJETOS COM TRATAMENTO DE
EXCEÇÕES**

SALVADOR/BA

2026

1 INTRODUÇÃO

Este trabalho aborda os conceitos de Programação Orientada a Objetos (POO), com foco em herança, uso de vetores de objetos e tratamento de exceções. O objetivo é desenvolver uma aplicação utilizando uma superclasse e subclasses.

2 DIAGRAMA DE HIERARQUIA

2.1 Classe Transporte (Superclasse)

A classe Transporte foi criada para armazenar os atributos comuns entre as classes Aviao e Navio. Atributos: capacidadeTanque, numeroPassageiros, preco.

Responsabilidade: Servir como base para herança, Centralizar dados comuns.

2.2 Classe avião:

A classe Aviao herda da classe Transporte e adiciona características específicas. Atributos: prefixo, dataRevisao.

Métodos: entrada(), exibição dos dados, ajuste do valor.

2.3 Classe Navio

A classe Navio também herda de Transporte, com atributos próprios. Atributos: nome, numeroTripulantes, dataLancamento

Métodos: entrada(), imprimir(), passageirosPorTripulantes()

3 OBJETIVO

O objetivo principal desse código é demonstrar, na prática, conceitos de herança, polimorfismo, encapsulamento e tratamento de exceções. O sistema realiza a entrada de dados pelo usuário, trata possíveis erros e exibe as informações armazenadas.

4 CÓDIGO

Classe Transporte

```
package model;

public class Transporte {
    protected int capacidadeTanque;
    protected int numeroPassageiros;
    protected double preco;

    public Transporte() {}

    public Transporte(int capacidadeTanque, int numeroPassageiros,
double preco) {
        this.capacidadeTanque = capacidadeTanque;
        this.numeroPassageiros = numeroPassageiros;
        this.preco = preco;
    }

    public int getCapacidadeTanque() { return capacidadeTanque; }
    public void setCapacidadeTanque(int capacidadeTanque) {
this.capacidadeTanque = capacidadeTanque; }

    public int getNumeroPassageiros() { return numeroPassageiros; }
    public void setNumeroPassageiros(int numeroPassageiros) {
this.numeroPassageiros = numeroPassageiros; }

    public double getPreco() { return preco; }
    public void setPreco(double preco) { this.preco = preco; }
}
```

Classe Navio

```
package model;

import java.util.Scanner;

public class Navio extends Transporte {
    private String nome;
```

```
private int numeroTripulantes;
private String dataLancamento;

public Navio() {}

public Navio(String nome) {
    this.nome = nome;
}

public Navio(String nome, int numeroTripulantes) {
    this.nome = nome;
    this.numeroTripulantes = numeroTripulantes;
}

    public Navio(String nome, int numeroTripulantes, String
dataLancamento) {
        this.nome = nome;
        this.numeroTripulantes = numeroTripulantes;
        this.dataLancamento = dataLancamento;
    }

    public Navio(String nome, int numeroTripulantes, String
dataLancamento,
                                int capacidadeTanque, int
numeroPassageiros, double preco) {
        super(capacidadeTanque, numeroPassageiros, preco);
        this.nome = nome;
        this.numeroTripulantes = numeroTripulantes;
        this.dataLancamento = dataLancamento;
    }

public void entrada(Scanner sc) {
    try {
        System.out.print("Nome: ");
        nome = sc.nextLine();
    }
```

```

        System.out.print("Data Lançamento: ");
        dataLancamento = sc.nextLine();

        System.out.print("Tripulantes: ");
        numeroTripulantes = sc.nextInt();

        System.out.print("Capacidade Tanque: ");
        capacidadeTanque = sc.nextInt();

        System.out.print("Passageiros: ");
        numeroPassageiros = sc.nextInt();

        System.out.print("Preço: ");
        preco = sc.nextDouble();

        sc.nextLine();

    } catch (Exception e) {
        System.out.println("Erro!");
        sc.nextLine();
    }
}

public void imprimir() {
    System.out.println("NAVIO");
    System.out.println("Nome: " + nome);
    System.out.println("Tripulantes: " +
numeroTripulantes);
    System.out.println("Data Lançamento: " +
dataLancamento);
    System.out.println("Capacidade Tanque: " +
capacidadeTanque);
    System.out.println("Passageiros: " +
numeroPassageiros);
    System.out.println("Preço: " + preco);
}

```

```

        public double passageirosPorTripulantes() {
            if (numeroTripulantes == 0) return 0;

            return (double) numeroPassageiros /
numeroTripulantes;
        }
    }
}

```

Classe Avião

```

package model;

import java.util.Scanner;

public class Aviao extends Transporte {
    private String prefixo;
    private String dataRevisao;

    public Aviao() {}

    public Aviao(String prefixo, String dataRevisao, int
capacidadeTanque, int numeroPassageiros, double preco) {
        super(capacidadeTanque, numeroPassageiros, preco);
        this.prefixo = prefixo;
        this.dataRevisao = dataRevisao;
    }

    public void entrada(Scanner sc) {
        try {
            System.out.print("Prefixo: ");
            prefixo = sc.nextLine();

            System.out.print("Data Revisão: ");
            dataRevisao = sc.nextLine();

            System.out.print("Capacidade Tanque: ");
            capacidadeTanque = sc.nextInt();

```

```

        System.out.print("Número Passageiros: ");
        numeroPassageiros = sc.nextInt();

        System.out.print("Preço: ");
        preco = sc.nextDouble();

        sc.nextLine(); // limpar buffer

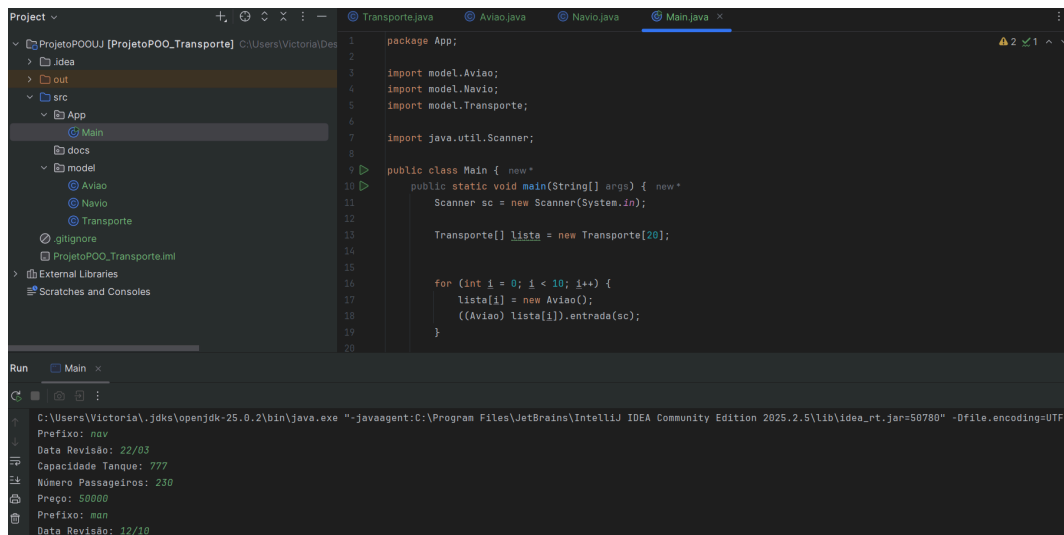
    } catch (Exception e) {
        System.out.println("Erro de entrada! Digite novamente.");
        sc.nextLine();
    }
}

public void imprimir() {
    System.out.println("AVIÃO");
    System.out.println("Prefixo: " + prefixo);
    System.out.println("Data Revisão: " + dataRevisao);
    System.out.println("Capacidade Tanque: " + capacidadeTanque);
    System.out.println("Passageiros: " + numeroPassageiros);
    System.out.println("Preço: " + preco);
}

public void reajustarPreco(double percentual) {
    preco += preco * (percentual / 100);
}
}

```

Saída



CONCLUSÃO

O desenvolvimento deste projeto permitiu aplicar, na prática, conceitos fundamentais da Programação Orientada a Objetos, como herança, encapsulamento, polimorfismo e tratamento de exceções.

Além disso, o tratamento de exceções garantiu maior resiliência da aplicação, evitando falhas durante a entrada de dados. Dessa forma, o projeto demonstrou a importância de uma boa modelagem e organização para o desenvolvimento de sistemas estruturados, reutilizáveis e de fácil manutenção.

REFERÊNCIAS

Documentação oficial da Oracle – Java SE.